

DISCIPLINA: MODELAGEM E PROJETO DE BANCO DE DADOS

Professor: Romes

Atividade Prática — Parte II

Criação de Esquema de Banco de Dados no MySQL Workbench

Aluno(a): _____ Turma: _____
Data: ____/____/____

1. Objetivo

Esta atividade tem como objetivo aplicar os conceitos de DDL (Data Definition Language) na construção de um esquema completo de banco de dados utilizando o MySQL Workbench. O aluno deverá criar um banco de dados com 10 tabelas relacionadas, contendo chaves primárias, chaves estrangeiras, restrições de integridade, índices e ao menos uma view, simulando um cenário real de sistema de informação.

2. Ferramentas e Pré-requisitos

- MySQL Server (versão 8.0 ou superior) instalado e em execução.
- MySQL Workbench instalado e conectado ao servidor local.
- Editor de SQL do Workbench para escrita e execução dos scripts DDL.
- Conhecimento prévio dos comandos: CREATE DATABASE, CREATE TABLE, ALTER TABLE, CREATE INDEX, CREATE VIEW e DROP TABLE.

3. Exemplo Orientador

Antes de iniciar a atividade, observe o exemplo abaixo de um esquema simplificado de uma livraria, contendo a criação do banco de dados, duas tabelas relacionadas e um índice. Use este exemplo como referência para o seu projeto.

3.1 Criação do Banco de Dados

```
-- Cria o banco de dados
CREATE DATABASE IF NOT EXISTS livraria_exemplo
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_general_ci;

-- Seleciona o banco de dados para uso
USE livraria_exemplo;
```

3.2 Criação das Tabelas

```
-- Tabela editora
CREATE TABLE editora (
  id_editora INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cidade VARCHAR(60),
  pais VARCHAR(40) DEFAULT 'Brasil'
) ENGINE=InnoDB;

-- Tabela livro com chave estrangeira para editora
CREATE TABLE livro (
```

```

    id_livro INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    ano_publicacao INT CHECK (ano_publicacao > 1500),
    preco DECIMAL(10,2) NOT NULL CHECK (preco >= 0),
    id_editora INT,
    CONSTRAINT fk_livro_editora
        FOREIGN KEY (id_editora) REFERENCES editora(id_editora)
        ON DELETE SET NULL
        ON UPDATE CASCADE
) ENGINE=InnoDB;

```

3.3 Criação de Índice e View

```

-- Índice para otimizar buscas por título
CREATE INDEX idx_livro_titulo ON livro (titulo);

-- View que apresenta livros com suas editoras
CREATE VIEW vw_livros_editora AS
SELECT l.id_livro, l.titulo, l.preco, e.nome AS editora
FROM livro l
LEFT JOIN editora e ON l.id_editora = e.id_editora;

```

4. Especificações do Projeto

Você deverá criar, no MySQL Workbench, um banco de dados chamado `clinica_medica`, contendo as 10 tabelas descritas a seguir. Todas as tabelas devem utilizar o mecanismo de armazenamento InnoDB e seguir as restrições de integridade indicadas.

4.1 Requisitos Gerais

1. Criar o banco de dados `clinica_medica` com charset `utf8mb4`.
2. Criar as 10 tabelas conforme as especificações abaixo, respeitando tipos, restrições e relacionamentos.
3. Toda tabela deve possuir chave primária (PRIMARY KEY) com AUTO_INCREMENT, exceto quando indicado.
4. Todas as chaves estrangeiras devem ser nomeadas no padrão `fk_<tabela_origem>_<tabela_destino>`.
5. Definir ao menos 3 índices adicionais em colunas estratégicas para otimização de consultas.
6. Criar pelo menos 1 view que envolva JOIN entre 3 tabelas distintas.
7. Salvar o script SQL completo em arquivo `.sql` e gerar também o modelo EER (Engenharia Reversa) no Workbench.

4.2 Tabelas do Banco de Dados `clinica_medica`

Tabela 1 — paciente

Atributo	Tipo	Restrição	Observação
id_paciente	INT	PK, AUTO_INCREMENT	Identificador único
nome	VARCHAR(100)	NOT NULL	Nome completo

Atributo	Tipo	Restrição	Observação
cpf	CHAR(11)	NOT NULL, UNIQUE	Apenas números
data_nascimento	DATE	NOT NULL	—
sexo	CHAR(1)	CHECK (sexo IN ('M','F','O'))	M, F ou O
telefone	VARCHAR(15)	—	—
email	VARCHAR(80)	UNIQUE	—

Tabela 2 — endereco

Atributo	Tipo	Restrição	Observação
id_endereco	INT	PK, AUTO_INCREMENT	—
id_paciente	INT	FK → paciente	ON DELETE CASCADE
logradouro	VARCHAR(120)	NOT NULL	—
numero	VARCHAR(10)	—	—
bairro	VARCHAR(60)	NOT NULL	—
cidade	VARCHAR(60)	NOT NULL	—
uf	CHAR(2)	NOT NULL	—
cep	CHAR(8)	NOT NULL	—

Tabela 3 — especialidade

Atributo	Tipo	Restrição	Observação
id_especialidade	INT	PK, AUTO_INCREMENT	—
nome	VARCHAR(60)	NOT NULL, UNIQUE	Ex.: Cardiologia
descricao	VARCHAR(200)	—	—

Tabela 4 — medico

Atributo	Tipo	Restrição	Observação
id_medico	INT	PK, AUTO_INCREMENT	—
nome	VARCHAR(100)	NOT NULL	—
crm	VARCHAR(15)	NOT NULL, UNIQUE	Registro profissional
id_especialidade	INT	FK → especialidade	NOT NULL
telefone	VARCHAR(15)	—	—

Atributo	Tipo	Restrição	Observação
email	VARCHAR(80)	UNIQUE	—

Tabela 5 — convenio

Atributo	Tipo	Restrição	Observação
id_convenio	INT	PK, AUTO_INCREMENT	—
nome	VARCHAR(80)	NOT NULL, UNIQUE	—
cnpj	CHAR(14)	NOT NULL, UNIQUE	—
telefone	VARCHAR(15)	—	—
ativo	BOOLEAN	DEFAULT TRUE	—

Tabela 6 — paciente_convenio

Atributo	Tipo	Restrição	Observação
id_paciente	INT	PK composta, FK → paciente	—
id_convenio	INT	PK composta, FK → convenio	—
numero_carteirinha	VARCHAR(30)	NOT NULL	—
data_adesao	DATE	NOT NULL	—

Tabela 7 — consulta

Atributo	Tipo	Restrição	Observação
id_consulta	INT	PK, AUTO_INCREMENT	—
id_paciente	INT	FK → paciente, NOT NULL	—
id_medico	INT	FK → medico, NOT NULL	—
data_hora	DATETIME	NOT NULL	—
status	VARCHAR(20)	DEFAULT 'AGENDADA'	AGENDADA/REALIZADA/ CANCELADA
valor	DECIMAL(10,2)	CHECK (valor >= 0)	—

Tabela 8 — prontuario

Atributo	Tipo	Restrição	Observação
id_prontuario	INT	PK, AUTO_INCREMENT	—
id_consulta	INT	FK → consulta, UNIQUE, NOT NULL	Relação 1:1 com consulta
sintomas	TEXT	—	—
diagnostico	TEXT	—	—
observacoes	TEXT	—	—
data_registro	DATETIME	DEFAULT CURRENT_TIMESTAMP	—

Tabela 9 — medicamento

Atributo	Tipo	Restrição	Observação
id_medicamento	INT	PK, AUTO_INCREMENT	—
nome	VARCHAR(100)	NOT NULL	—
principio_ativo	VARCHAR(100)	NOT NULL	—
fabricante	VARCHAR(80)	—	—
preco_unitario	DECIMAL(10,2)	CHECK (preco_unitario >= 0)	—

Tabela 10 — prescricao

Atributo	Tipo	Restrição	Observação
id_prescricao	INT	PK, AUTO_INCREMENT	—
id_prontuario	INT	FK → prontuario, NOT NULL	—
id_medicamento	INT	FK → medicamento, NOT NULL	—
dosagem	VARCHAR(50)	NOT NULL	Ex.: 1 comp. 8/8h
duracao_dias	INT	CHECK (duracao_dias > 0)	—
observacoes	VARCHAR(200)	—	—

5. Relacionamentos Esperados

- paciente 1 : N endereco
- paciente N : M convenio (via paciente_convenio)

- especialidade 1 : N medico
- paciente 1 : N consulta
- medico 1 : N consulta
- consulta 1 : 1 prontuario
- prontuario 1 : N prescricao
- medicamento 1 : N prescricao

6. Critérios de Avaliação e Entrega

8. Script SQL completo (.sql) com todos os comandos CREATE DATABASE, CREATE TABLE, ALTER TABLE (se necessário), CREATE INDEX e CREATE VIEW.
9. Arquivo de modelo do Workbench (.mwb) com o diagrama EER gerado por engenharia reversa.
10. Captura de tela do diagrama EER mostrando as 10 tabelas e seus relacionamentos.
11. Documento contendo o nome do aluno, descrição das 3 views/índices criados e justificativa das escolhas de tipos e restrições.

6.1 Pontuação Sugerida

- Criação correta das 10 tabelas com tipos e restrições adequados — 4,0 pontos
- Definição correta das chaves estrangeiras e integridade referencial — 2,0 pontos
- Criação de índices e views conforme solicitado — 1,5 pontos
- Diagrama EER gerado no Workbench — 1,5 pontos
- Organização, padronização e documentação do código — 1,0 ponto

7. Dicas Finais

- Comece criando o banco e as tabelas independentes (sem FK). Depois crie as tabelas que dependem delas.
- Use o recurso Reverse Engineer do Workbench (Database → Reverse Engineer) para gerar automaticamente o diagrama EER após executar seu script.
- Sempre teste o script em um banco limpo para garantir que ele executa de ponta a ponta sem erros.
- Use comentários no SQL (-- ou /* */) para documentar suas decisões de modelagem.

Bom projeto e boa modelagem!